

1. Make the page a StatefulWidget/ConsumerStatefulWidget
2. In initState, call the remote API
3. Display CircularProgressIndicator
4. Display Data

! INFO

`FutureProvider` does not offer a way of directly modifying the computation after a user interaction. It is designed to solve simple use-cases.

For more advanced scenarios, consider using [AsyncNotifierProvider](#).

```
void invalidate(  
    ProviderOrFamily provider  
)
```

```
@useResult  
T refresh<T>(   
    Refreshable<T> provider  
)
```

```
void invalidate(  
    ProviderOrFamily provider  
)
```

Invalidates the state of the provider, causing it to refresh.

As opposed to [refresh](#), the refresh is not immediate and is instead delayed to the next read or next frame.

Calling [invalidate](#) multiple times will refresh the provider only once.

Calling [invalidate](#) will cause the provider to be disposed immediately.

If used on a provider which is not initialized, this method will have no effect.

```
T refresh<T>(
    Refreshable<T> provider
)
```

Forces a provider to re-evaluate its state immediately, and return the created value.

Writing:

```
final newValue = ref.refresh(provider);
```

is strictly identical to doing:

```
ref.invalidate(provider);
final newValue = ref.read(provider);
```

If you do not care about the return value of [refresh](#), use [invalidate](#) instead. Doing so has the benefit of:

- making the invalidation logic more resilient by avoiding multiple refreshes at once.
- possibly avoids recomputing a provider if it isn't needed immediately.

This method is useful for features like "pull to refresh" or "retry on error", to restart a specific provider.